

Lets Learn Claude Code (LLCC)

Yogesh Haribhau Kulkarni

Choosing the Right Model

Claude Code ships three model tiers. Use the right one:

Model	Best For	Avoid For
Haiku 4.5	commands, exploration, quick edits	Deep reasoning
Sonnet 4.6 (default)	Daily coding, refactors, implementation	Trivial lookups
Opus 4.6+	Architecture, hard trade-off calls	Repetitive work

Setting model:

```
/model claude-opus-4-6 # inside a session
claude --model claude-opus-4-6 # Or launch

# Per-project default in .claude/settings.json:
{ "model": "claude-sonnet-4-6" }
```

Stage 1: Project Bootstrap

Run /init: Generate CLAUDE.md

- Start `claude` and at the prompt say `/init` as shown below
- Claude asks clarifying questions about the project: answer them, then approve the file creation.

```
/init

## What /init generates:
CLAUDE.md           # Project intelligence file
.claude/settings.json # Permissions, model config

## Resulting CLAUDE.md (edit to match below):
---
# CLAUDE.md

## Project: stlinspector
STL/STEP geometry inspector and validator (PoC).

## Tech Stack
- Python 3.11, trimesh, numpy
- Streamlit + plotly (3D preview), Rich (CLI)

## Build Commands
- Install: 'pip install trimesh numpy streamlit
  plotly pytest rich'
- Test:   'pytest tests/ -v'
---
## Commit immediately:
```

```
git add .
git commit -m "chore: added CLAUDE.md"
```

Stage 2: Specifications

Specs Subagent Writes the Project Spec

- Crate a **specs** subagent that keeps spec-writing out of your main context.
- `mkdir -p .claude/agents`
- `cat > .claude/agents/specs.md` as below

```
---
name: specs
description: >
  Technical specification writer. Produces
  structured
  Markdown specs from feature descriptions.
  Read-only.
model: claude-opus-4-6
allowed-tools: Read, Write, Glob
---
Produce a Markdown spec covering: Overview, Goals,
Non-Goals, Functional Requirements, Data Models,
API/CLI Contracts, Open Questions. Save to
docs/specs/<feature>.md.

## Invoke it via following prompt:
---
Write a technical spec for stlinspector.
Cover:
- core.py: load_mesh(path), InspectionReport
  dataclass
  (bbox, volume, surface_area, triangle_count,
  issues)
- Validation rules: not-watertight, non-manifold
  edges,
  degenerate faces
- cli.py: python cli.py <file> [--report out.json]
  (no console-script entry point in this pass)
- app.py: upload -> 3D preview -> report table
Save to docs/specs/stlinspector-spec.md
---
git add docs/
git commit -m "docs: added stlinspector spec"
```

Demo

STL Inspector App Workflow

Use-case

A STEP/STL geometry inspector & validator, `stlinspector`, with a CLI and a Streamlit app. Meshes exported from CAD/CAM tools can silently carry bad geometry – non-watertight shells, non-manifold edges, degenerate faces – that only surfaces later, as a failed 3D print or a broken CAM tool-path. `stlinspector` catches these issues right after export, with both a scriptable CLI (for CI pipelines) and a Streamlit app (for a quick visual check). This workshop builds it end-to-end in a single solo walkthrough, using Claude Code’s structural primitives – commands, subagents, skills, and hooks – on top of the standard chat loop.

Claude Code Customization, at a Glance

Four ways to extend Claude Code beyond the standard chat loop, used throughout this workshop:

Primitive	Invoked via	Purpose
Commands	Typed slash command, e.g. <code>/add-check</code>	Repeatable, team-shared workflow saved as a prompt
Subagents	Delegated by name, e.g. <code>specs</code> , <code>qa</code>	Isolate a task’s context; only its summary returns
Skills	Auto-loaded when the task matches its description	Lazy-loaded domain knowledge, not in every session
Hooks	Fire automatically on lifecycle events	Deterministic guardrail – always runs, not a request

Setup

Pre-requisites

Essential Steps/Checklist:

- Create an **empty folder** and switch to it
- `mkdir stlinspector && cd stlinspector`
- Conda installed: `conda --version`
- Claude Code installed: `claude doctor`
- Have Claude Pro/Max or API key
- Create the conda env and install demo deps

```
conda create -n stlinspector python=3.11 -y
conda activate stlinspector

pip install trimesh numpy streamlit plotly pytest
rich
```

Stage 3: Architecture Design

Design, Then Scaffold

Plan mode is **read-only**: no edits, no bash, and is perfect for exploring the module split before committing to it. Shift+Tab # Enter Plan mode

```
-- Prompt --
Read docs/specs/stlinspector-spec.md, then design
the package layout for stlinspector:
- src/{core,cli,app,config}.py
- Data flow: file -> core.load_mesh() -> mesh ->
  core.inspect_mesh() -> InspectionReport
- What core.py must NOT depend on (no Streamlit
  imports)
Show the plan before creating any files.
--
## Approve ("Yes, auto-accept edits")

\lstineline|Shift+Tab # Enter Normal mode|

-- Prompt --
Create the stlinspector scaffold with stub methods:
- src/core.py (stub: load_mesh(path),
  inspect_mesh(mesh))
- src/cli.py (stub: main())
- src/app.py (stub: Streamlit skeleton)
- src/config.py (stub: Config dataclass)
- tests/__init__.py, tests/conftest.py,
  tests/test_core.py
- README.md

---

git add . && git commit -m "chore: scaffold
stlinspector"
/cost # Check token usage so far
```

Stage 4: Implementation

Implement core.py: the Inspection Engine

```
@file:src/core.py
@file:docs/specs/stlinspector-spec.md
Implement core.py:
- load_mesh(path: str) -> trimesh.Trimesh
```

```
- inspect_mesh(mesh: trimesh.Trimesh) ->
  InspectionReport, where InspectionReport is a
  dataclass with: bounding_box, volume,
  surface_area, triangle_count, issues:
  list[str]
- Issue checks:
  - "not_watertight" if not mesh.is_watertight
  - "non_manifold_edges" if any edge shared by > 2
    faces
  - "degenerate_faces" if any face area < 1e-8
- Full type hints, Google-style docstrings
- Raise FileNotFoundError / ValueError with clear
  messages

---
git add src/core.py
git commit -m "feat: implement inspection engine"
```

Implement cli.py and app.py

```
## CLI:
@file:src/cli.py
Implement the CLI (run via 'python cli.py', no
console-script entry point in this pass):
- load_mesh(path) then inspect_mesh(mesh)
  [--report out.json] via argparse
- Use Rich Table to print
  bbox/volume/area/triangle_count
- Print issues in red if any, green "All checks
  passed"
- Exit code 1 if any issue found (CI-friendly)
- JSON is the only report format (via --report
  out.json)
  -- this feeds the inspection-report-summary
  skill later

---
## Streamlit:
@file:src/app.py
Implement the Streamlit app:
- st.file_uploader restricted to .stl
- Save upload to a temp file, call
  core.load_mesh() then core.inspect_mesh()
- 3D preview via plotly go.Mesh3d
  (mesh.vertices/faces)
- st.metric tiles for
  bbox/volume/area/triangle_count
- st.error per issue, st.success if none

---
## Run and verify (no entry point installed yet):
python src/cli.py tests/fixtures/box.stl --report
out.json
streamlit run src/app.py
```

```
---
git add src/ && git commit -m "feat: implement CLI
and Streamlit app"
```

Custom Commands & Skills

Create the /add-check Custom Command

- If you would repeat the same “add a validation rule” prompt more than once, make it a command.
- mkdir -p .claude/commands
- Write .claude/commands/add-check.md as below

```
---
description: >
  Scaffold a new geometry validation rule in
  core.py,
  wired into InspectionReport.issues, plus a
  matching
  synthetic-fixture test. Pass the rule name as
  argument.
allowed-tools: Read, Edit, Write, Bash
---
Add a new validation check named "$ARGUMENTS" to
src/core.py:
1. Add a private _check_$ARGUMENTS(mesh) -> bool
  helper
2. Wire it into inspect_mesh(), appending
  "$ARGUMENTS" to
  issues when it returns True
3. Add a synthetic-mesh test in tests/test_core.py
  that
  triggers the new check
4. Run pytest tests/ -v and report the result

## Usage:
/add-check thin_walls
```

Create the inspection-report-summary Skill

- Skills are lazy-loaded: Claude pulls this in only when the task matches, unlike CLAUDE.md which loads every session.
- A skill cannot do geometry math itself – core.py/trimesh already did that. What it can do well is turn a report into a stakeholder-ready summary.
- mkdir -p .claude/skills/inspection-report-summary
- Write SKILL.md in that folder, as below

```

---
name: inspection-report-summary
description: >
  Summarizes an stlinspector InspectionReport
  (JSON)
  for a non-technical stakeholder. Use when asked
  to
  "summarize", "explain", or "triage" an inspection
  report.
---

## Input
Read the JSON report written by --report (e.g.
out.json,
or the path given): bbox, volume, surface_area,
triangle_count, and a list of issues (if any).

## Output
- One-paragraph plain-English verdict: PASS / FAIL
  and why
- If issues present, rank by severity:
  1. not_watertight (blocks 3D printing / CAM)
  2. non_manifold_edges (blocks CAM toolpath
  generation)
  3. degenerate_faces (usually cosmetic, low risk)
- Suggest the next concrete action per issue found
  (e.g. "re-export from CAD with tighter
  tessellation")

## Generate a report to summarize (if out.json
isn't already present):
python src/cli.py src/Box.stl --report out.json

## Usage:
Summarize out.json for a non-technical reviewer

```

QA Workflows

QA Subagent

- Synthetic Fixtures, No Sample Files
- Write .claude/agents/qa.md as below.
- Use the qa subagent to write tests/conftest.py with fixtures: valid_box_mesh, non_watertight_mesh, degenerate_face_mesh. Then write tests/test_core.py covering all three cases against core.inspect_mesh().

```

description: >
  Generates pytest tests using synthetic trimesh
  fixtures
  (no real STL files). Use when adding tests for
  core.py.
model: claude-sonnet-4-6
allowed-tools: Read, Write, Bash
---
Generate fixtures with trimesh.creation (box,
icosphere)
for the valid case, and hand-built meshes (remove
a face for
non-watertight, zero-area triangle for degenerate)
for
the failing cases. Assert on InspectionReport
fields
and on report.issues membership.

## Usage:
Use the qa subagent to write tests/conftest.py with
fixtures: valid_box_mesh, non_watertight_mesh,
degenerate_face_mesh. Then write tests/test_core.py
covering all three cases against
core.inspect_mesh().

```

Run Tests and Check Coverage

```

pytest tests/ -v --tb=short

pytest tests/ --cov=src --cov-report=term-missing

---
# If a test fails, hand Claude the traceback
directly:
---
@file:src/core.py
test_degenerate_face_mesh fails with:
AssertionError: 'degenerate_faces' not in
report.issues
Here is the fixture: [paste]. Fix the check or the
fixture.
---
git add tests/
git commit -m "test: add synthetic-fixture test
suite"

```

Hooks: Deterministic Guardrails

Wire Up Hooks in .claude/settings.local.json

- Hooks are deterministic: they always fire, unlike a prompt-based instruction.
- What this buys:
 - PostToolUse: every edited file is auto-formatted
 - Stop: the suite must pass before Claude can finish
 - PreToolUse: destructive bash commands are blocked even if Claude "forgets", no prompt can override this

```

{
  "permissions": {
    "allow": [
      :
    ]
  },
  "hooks": {
    "PostToolUse": [{
      "matcher": "Edit|Write",
      "hooks": [{ "type": "command",
        "command": "FILE=$(cat | jq -r
        '.tool_input.file_path'); ruff format
        \"$FILE\" 2>/dev/null || true"}]
    }],
    "Stop": [{
      "hooks": [{ "type": "command",
        "command": "cd \"$CLAUDE_PROJECT_DIR\" &&
        pytest tests/ -q --tb=short || echo
        '{\"decision\": \"block\", \"reason\":
        \"tests failing\"}'"}]
    }],
    "PreToolUse": [{
      "matcher": "Bash",
      "hooks": [{ "type": "command",
        "command": "INPUT=$(cat); CMD=$(echo $INPUT |
        jq -r '.tool_input.command'); echo \"$CMD\" |
        grep -qE 'rm -rf|DROP TABLE' && echo
        'Blocked: destructive command' >&2 && exit 2
        || exit 0"}]
    }
  ]
}

```

Run End-to-End and Recap

What we covered: bootstrap → spec → architecture → implementation → commands & skills → QA subagent → hooks – everything needed to ship stlinspector end-to-end.

```

python src/cli.py tests/fixtures/box.stl --report
out.json
# Expected: Rich table with
bbox/volume/area/triangle_count

```

```

---
name: qa

```

```
# Writes out.json (the only report format)

streamlit run src/app.py
# Expected: browser opens at http://localhost:8501
# Upload an STL, see 3D preview + report

pytest tests/ -v
# All synthetic-fixture tests pass
```

Best Practices & Token Management

Token Budgeting and Model Choice

- Every input *and* output token is billed: unfocused sessions on Sonnet/Opus cost more than necessary.
- Bloating context buries signal under failed attempts and irrelevant files; a smaller, focused context produces sharper output.
- **Rule of thumb:** default to Sonnet; drop to Haiku for subagents doing **qa** triage runs; promote to Opus only for real architecture trade-offs.

```
/model claude-haiku-4-5      # Cheap subagent
work
/cost                        # Check spend
so far
```

Plan Mode and Breaking Tasks Down

Plan mode (Shift+Tab) is read-only: exploring a wrong approach costs almost nothing; implementing it wrong costs a redo. Stay in **normal mode** (not auto-accept) for multi-step work; only auto-accept mechanical edits already reviewed in Plan mode.

```
I want to add a wall-thickness check to
stlinspector.
Break this into numbered subtasks and show me the
list
first. Do NOT start until I confirm each step.

## TODO Write tracks progress automatically as steps
## complete, one step in progress at a time, no
## parallel context drift.
```

Context Hygiene

- **/compact:** summarise older turns once **/cost** shows context past ~50% of the window.
- **@file:** scoping: load only the file(s) the current step needs: loading the whole **src/** tree for a one-file fix is the most common waste.
- Subagents (**specs**, **qa**) run in isolated contexts: only their summary returns to the main session.
- Skills are lazy-loaded: **inspection-report-summary** only enters context when the task matches.
- **/clear:** nuclear option, only for a genuinely unrelated new task.

```
@file:src/core.py  # Not the whole src/ tree
/compact
```

Structure Beats Prompting Skill

Everything used in this workshop is a structural layer, not a clever prompt:

- **CLAUDE.md:** execution memory, loaded every session (WHY/WHAT/HOW, kept short)
- **.claude/commands/:** repeated workflows as **/add-check-style** commands, committed to Git for the whole team
- **.claude/rules/:** path-scoped constraints near sharp edges (e.g. **core.py** numerics)
- **.claude/skills/:** lazy-loaded domain knowledge (**inspection-report-summary**)
- Hooks: deterministic guardrails, not requests

Prompts are temporary. Structure is permanent: it is what let Stages 5 to 8 of this workshop run with almost no re-explaining.

Session Continuity Across Days

A workshop like this one often spans multiple sittings: treat sessions as resumable, not disposable.

```
/rename stlinspector-implementation # Name
current session
claude --resume stlinspector-implementation
claude -c # Resume
most recent
```

```
## Or: write progress to a plan file Claude can
re-read:
Write our progress to .claude/plans/current.md as a
numbered checklist with [x]/[ ] and the next step
noted
at the top.
```

Scaling Up: Large CAD/CAM Codebases

stlinspector is a small PoC. Real CAD/CAM systems (geometry kernels, STEP/IGES parsers, GUI layers) can be millions of lines, so the same habits matter more, not less:

- **Module-scoped subagents:** one for the geometry kernel, one for file-format parsers, one for the GUI, never one subagent with the whole repo in scope.
- **Directory-scoped rules files:** e.g. **.claude/rules/kernel-rules.md** with paths: **["kernel/**"]**, so numerics constraints load only when kernel files are touched.
- **@file: precision:** never ask Claude to “read the parser”, instead reference the exact translation-unit or module; a full STEP/IGES parser can be tens of thousands of lines.

Scaling Up: Output Token Discipline

- **Ask for diffs, not rewrites:** “show the changed lines” beats “rewrite the file”: kernel files are often thousands of lines; a full rewrite burns output tokens and risks silently dropping code.
- **Synthetic fixtures over real files:** as in this workshop, use **trimesh.creation** primitives, not multi-MB STEP files, as test/context input.
- **Plan mode is mandatory before touching kernel/numerics code:** a wrong edit to tolerance-sensitive geometry code is expensive to detect and worse to revert.
- **One validation rule (or one kernel change) per request:** mirrors the **/add-check** pattern; batching multiple kernel changes in one prompt compounds risk and review cost.